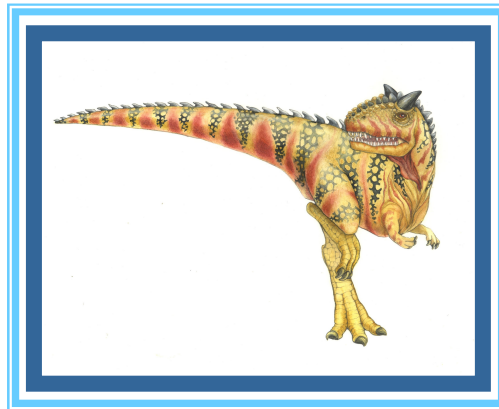


Chapter 8: Deadlocks





Outline

- **System Model**
- **Deadlock Characterization**
- **Methods for Handling Deadlocks**
- **Deadlock Prevention**
- **Deadlock Avoidance**
- **Deadlock Detection**
- **Recovery from Deadlock**





Chapter Objectives

- Illustrate how deadlock can occur when mutex locks are used
- Define the four necessary conditions that characterize deadlock
- Identify a deadlock situation in a resource allocation graph
- Evaluate the four different approaches for preventing deadlocks
- Apply the banker's algorithm for deadlock avoidance
- Apply the deadlock detection algorithm
- Evaluate approaches for recovering from deadlock





System Model

- System consists of resources
- Resource types $R_1, R_2, \dots, R_m$
 - *CPU cycles, memory space, I/O devices*
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**





Deadlock with Semaphores

- Data:
 - A semaphore s_1 initialized to 1
 - A semaphore s_2 initialized to 1
- Two threads T_1 and T_2
- T_1 :
 - `wait(s1)`
 - `wait(s2)`
- T_2 :
 - `wait(s2)`
 - `wait(s1)`





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one thread at a time can use a resource
- **Hold and wait:** a thread holding at least one resource is waiting to acquire additional resources held by other threads
- **No preemption:** a resource can be released only voluntarily by the thread holding it, after that thread has completed its task
- **Circular wait:** there exists a set $\{T_0, T_1, \dots, T_n\}$ of waiting threads such that T_0 is waiting for a resource that is held by T_1 , T_1 is waiting for a resource that is held by T_2 , ..., T_{n-1} is waiting for a resource that is held by T_n , and T_n is waiting for a resource that is held by T_0 .





Resource-Allocation Graph

A set of vertices V and a set of edges E .

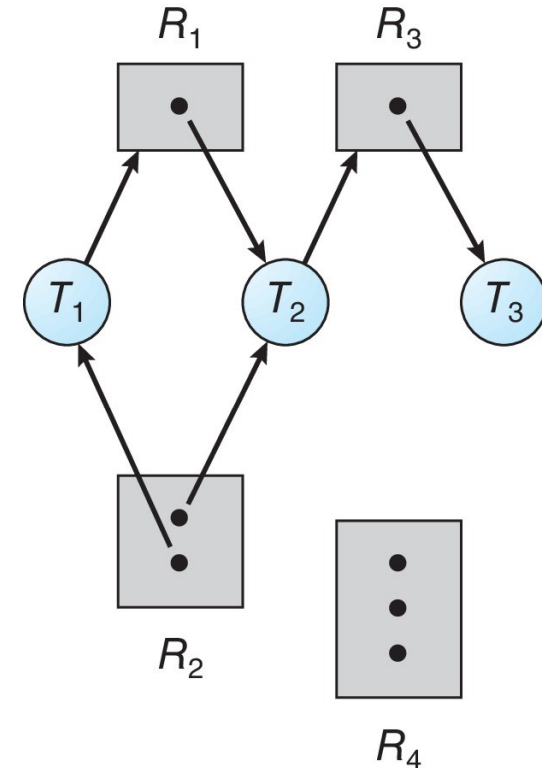
- V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set consisting of all the threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $T_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow T_i$





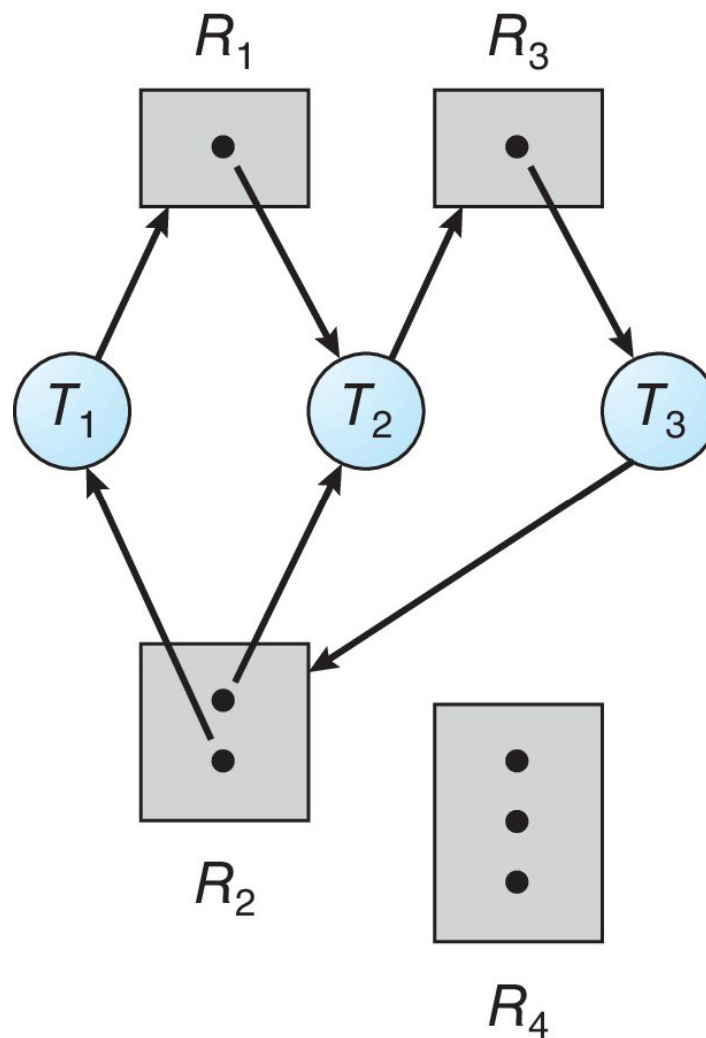
Resource Allocation Graph Example

- One instance of R_1
- Two instances of R_2
- One instance of R_3
- Three instance of R_4
- T_1 holds one instance of R_2 and is waiting for an instance of R_1
- T_2 holds one instance of R_1 , one instance of R_2 , and is waiting for an instance of R_3
- T_3 is holds one instance of R_3



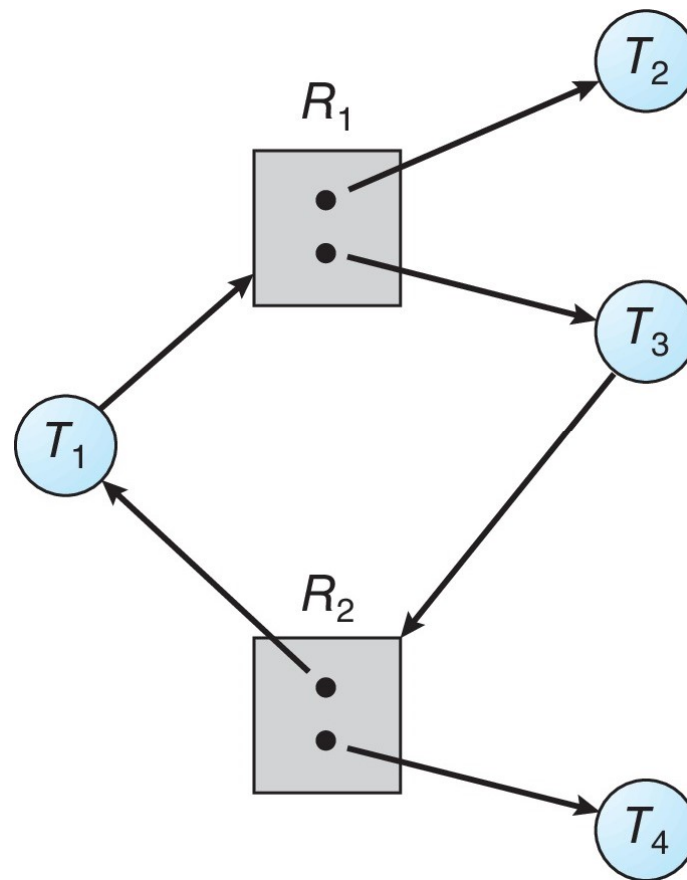


Resource Allocation Graph with a Deadlock





Graph with a Cycle But no Deadlock





Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock

